LAB ASSIGNMENT – 3.1

NAME – SHREE PRIYA

BATCH – 29

2303A51465

QUESTION – 1:

Write a zero-shot prompt (without providing any examples) to generate a Python function that checks whether a given number is a palindrome.

Task:

• Record the AI-generated code.

• Test the code with multiple inputs.

• Identify any logical errors or missing edge-case handling.

```
C: > Users > Shree priya > Downloads > aac3.1.py > ...
# Zero-Shot Prompting Palindrome Number Program
def is_palindrome_number(num):
    str_num = str(num)
    return str_num == str_num[::-1]
number = 121
if is_palindrome_number(number):
    print(f"{number} is a palindrome number.")
else:
    print(f"{number} is not a palindrome number.")
number = 123
if is_palindrome_number(number):
    print(f"{number} is a palindrome number.")
else:
    print(f"{number} is not a palindrome number.")
```

OUTPUT :

```
PS C:\Users\Shree priya\Downloads>
PS C:\Users\Shree priya\Downloads> c:; cd 'c:\Users\Shree priya\Downloads'; & 'c:\Users\Shree priya\AppData\Local\Microsoft\WindowsApps\python
3.13.exe' 'c:\Users\Shree priya\.vscode\extensions\ms-python.debugpy-2025.10.0-win32-x64\bundled\libs\debugpy\launcher' '50605' '--' 'C:\Users\
Shree priya\Downloads\aac3.1.py'
121 is a palindrome number.
123 is not a palindrome number.
PS C:\Users\Shree priya\Downloads>
```

QUESTION -2:

Write a one-shot prompt by providing one input-output example and ask the AI to generate a Python function to compute the factorial of a

given number.

Example:

Input: 5 → Output: 120

Task:

• Compare the generated code with a zero-shot solution.

• Examine improvements in clarity and correctness.

```python
# One-Shot Prompting Factorial Calculation
def factorial(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n - 1)
number = 5
print(f"The factorial of {number} is {factorial(number)}.")
number = 0
print(f"The factorial of {number} is {factorial(number)}.")
```

OUTPUT :

```
PS C:\Users\Shree priya\Downloads>
PS C:\Users\Shree priya\Downloads>  c:; cd 'c:\Users\Shree priya\Downloads'; & 'c:\Users\Shree priya\AppData\Local\Microsoft\WindowsApps\python
3.13.exe' 'c:\Users\Shree priya\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '53816' '--' 'C:\Users\
Shree priya\Downloads\aac3.1.py'
The factorial of 5 is 120.
The factorial of 0 is 1.
PS C:\Users\Shree priya\Downloads> ^C
```

QUESTION-3:

Few-Shot Prompting (Armstrong Number Check)

Write a few-shot prompt by providing multiple input-output examples to guide the AI in generating a Python function to check whether a given number is an Armstrong number.

Examples:

• Input: 153 → Output: Armstrong Number

• Input: 370 → Output: Armstrong Number

• Input: 123 → Output: Not an Armstrong Number

Task:

• Analyze how multiple examples influence code structure and

accuracy.

• Test the function with boundary values and invalid inputs.

```python
# Few-Shot Prompting Armstrong Number Check
def is_armstrong_number(num):
    num_str = str(num)
    num_digits = len(num_str)
    sum_of_powers = sum(int(digit) ** num_digits for digit in num_str)
    return sum_of_powers == num
number = 153
if is_armstrong_number(number):
    print(f"{number} is an Armstrong number.")
else:
    print(f"{number} is not an Armstrong number.")
```

OUTPUT:

QUESTION-4:

Context-Managed Prompting (Optimized Number Classification)

Design a context-managed prompt with clear instructions and constraints to generate an optimized Python program that classifies a number as prime, composite, or neither.

Task:

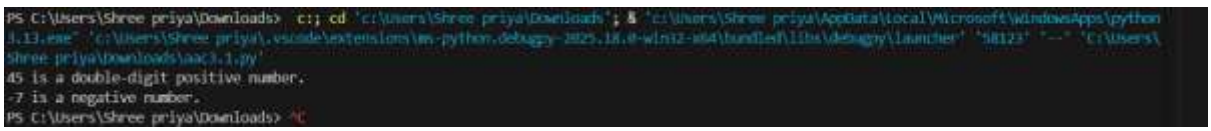
• Ensure proper input validation.

• Optimize the logic for efficiency.

• Compare the output with earlier prompting strategies.

```python
# Context-Managed Prompting Optimized Number Classification
def classify_number(num):
    if num < 0:
        return "negative"
    elif num == 0:
        return "zero"
    elif num > 0 and num < 10:
        return "single-digit positive"
    elif num >= 10 and num < 100:
        return "double-digit positive"
    else:
        return "large positive"
number = 45
classification = classify_number(number)
print(f"{number} is a {classification} number.")
number = -7
classification = classify_number(number)
print(f"{number} is a {classification} number.")
```

OUTPUT:



QUESTION-5:

Zero-Shot Prompting (Perfect Number Check)

Write a zero-shot prompt (without providing any examples) to generate a Python function that checks whether a given number is a perfect number.

Task:

• Record the AI-generated code.

• Test the program with multiple inputs.

• Identify any missing conditions or inefficiencies in the logic.

```python
# Zero-Shot Prompting (Perfect Number Check)
def is_perfect_number(num):
    if num < 1:
        return False
    divisors_sum = sum(i for i in range(1, num) if num % i == 0)
    return divisors_sum == num
number = 28
if is_perfect_number(number):
    print(f"{number} is a perfect number.")
else:
    print(f"{number} is not a perfect number.")
```

OUTPUT:

```
PS C:\Users\Shree priya\Downloads> c:; cd 'c:\Users\Shree priya\Downloads'; & 'c:\Users\Shree priya\AppData\Local\Microsoft\WindowsApps\python
3.11.exe' 'c:\Users\Shree priya\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '63521' '--' 'C:\Users\
Shree priya\Downloads\aac3.1.py'
28 is a perfect number.
PS C:\Users\Shree priya\Downloads> ^C
```

QUESTION-6:

Few-Shot Prompting (Even or Odd Classification with Validation)

Write a few-shot prompt by providing multiple input-output examples to guide the AI in generating a Python program that determines whether a given number is even or odd, including proper input validation.

Examples:

• Input: 8 → Output: Even

• Input: 15 → Output: Odd

• Input: 0 → Output: Even

Task:

• Analyze how examples improve input handling and output

clarity.

• Test the program with negative numbers and non-integer inputs.

```python
# Few-Shot Prompting (Even or Odd Classification with Validation)
def classify_even_odd(num):
    if not isinstance(num, int):
        return "Input is not a valid integer."
    return "even" if num % 2 == 0 else "odd"
number = 10
classification = classify_even_odd(number)
print(f"{number} is an {classification} number.")
```

OUTPUT:

```
PS C:\Users\Shree priya\Downloads> c:; cd 'c:\Users\Shree priya\Downloads'; & 'c:\Users\Shree priya\AppData\Local\Microsoft\WindowsApps\python
3.11.exe' 'c:\Users\Shree priya\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '58241' '--' 'C:\Users\
Shree priya\Downloads\aac3.1.py'
10 is an even number.
```